

FRIDAY, AUGUST 14TH, 2026

OPERATIONS RESEARCH

The Simplex Method and the Geometry of Optimization

Why the best answer to a linear problem always sits at a corner, how Dantzig taught a machine to walk between corners, and why an algorithm with a catastrophic worst case has never actually failed in practice.

3,062 words · 16-minute read

In 1945 the Chicago economist George Stigler set himself a whimsical challenge. Suppose you want to feed a 154-pound, moderately active man for a year at the lowest possible cost while still meeting every nutritional standard the National Research Council had established — calories, protein, calcium, iron and five vitamins, nine constraints in all, drawn from a list of 77 candidate foods. Stigler had no algorithm. He had a pencil, a table of nutrient contents, and a knack for informed trial and error. After days of hand iteration he arrived at a diet of wheat flour, evaporated milk, cabbage, spinach and dried navy beans, costing \$39.93 a year in 1939 prices [1]. He could not prove it was the cheapest combination available; he could only report that he had failed to find anything better.

Seven years later George Dantzig ran the identical 77-food, nine-constraint problem through a method he had invented in 1947 for an entirely unrelated purpose, and the answer came back at \$39.69 — Stigler's guesswork had missed the true minimum by 24 cents a year [1], [4]. The interesting part is not the diet, and not even the smallness of the gap. It is that the machine could certify its answer while the economist could only vouch for his diligence. Something about the structure makes certainty cheap once you look at the problem correctly. That something is geometric, and it is the real subject here: the set of all admissible diets is a solid with flat faces and finitely many corners, and the cheapest diet is always sitting on one of those corners. Everything else — Dantzig's algorithm, its spectacular commercial career, its embarrassing worst case — follows from working out what that fact lets you do.

From Air Force Scheduling to a General Theory

Dantzig came to the problem sideways. He had studied statistics at Berkeley under Jerzy Neyman, and once mistook two unsolved problems on a blackboard for a homework assignment, solving both before learning they were open research questions [7]. During the Second World War he ran the Combat Analysis Branch of Army Air Forces Headquarters Statistical Control, coordinating the movement of personnel, training programs, equipment and supplies at a scale nobody had previously tried to plan by hand [2], [3]. In 1947 he was posted to the Pentagon as mathematical advisor to Project SCOOP — Scientific Computation of Optimum Programs — with an explicit mandate to mechanize military

planning [3]. The Air Force used “program” the way a manager uses “plan”: a program was a proposed schedule of activities. Dantzig’s assignment was to compute the best program rather than merely a workable one, subject to scarce aircraft, fuel, personnel-hours and funding.

What he produced in the autumn of 1947 was a way of writing any such planning problem as a system of linear inequalities, together with a step-by-step procedure — the simplex algorithm — for walking through the space of feasible plans until the best one was reached. Tjalling Koopmans, visiting the RAND Corporation the following year, suggested calling the subject “linear programming,” borrowing the Air Force’s own vocabulary [2].

He was not first, though he had no way of knowing it. In 1939 a young Leningrad mathematician named Leonid Kantorovich was approached by engineers at a plywood trust with an allocation problem: how to assign several types of cutting machine, of differing productivity, across several types of wood, so as to maximize total output [5]. Kantorovich solved it with what he called “resolving multipliers,” a version of duality derived from first principles eight years before Dantzig and independently of any Western literature. He wrote it up as *Mathematical Methods in the Organization and Planning of Production*, printed in a run of a thousand copies [5]. It went almost nowhere, and not by accident. Soviet orthodoxy held that value derived from labor, not from a calculus that assigned “objectively determined valuations” to scarce inputs; one economist who heard Kantorovich present in 1943 reportedly warned him that talk of “the optimum” sat dangerously close to the work of “the fascist Pareto” [5]. He kept working quietly for two more decades. By the time the West rediscovered him in the 1950s, mathematics that had been politically radioactive in Moscow was running procurement schedules for the United States Air Force. In 1975 Kantorovich and Koopmans shared the Nobel Memorial Prize in Economic Sciences for their work on the optimal allocation of resources, and Koopmans reportedly offered to share part of his prize money with Dantzig, on the grounds that the algorithm belonged to him and not to an economics committee [5].

Year	Milestone
1939	Kantorovich solves the plywood-trust problem in Leningrad [5]
1947	Dantzig develops the simplex method for Project SCOP [2], [3]
1948	Koopmans coins “linear programming” at RAND [2]
1952	Dantzig confirms Stigler’s diet is off by 24 cents [1], [4]
1975	Kantorovich and Koopmans share the Nobel Memorial Prize [5]
1979	Khachiyan proves polynomial-time LP is possible [8]
1984	Karmarkar makes polynomial-time LP practical [6]

Why the Optimum Sits at a Corner

Stripped of history, a linear program is a spare object. Choose values for a vector of decision variables $\mathbf{x} \in \mathbb{R}^n$ to maximize a linear objective subject to linear inequalities:

$$\begin{array}{ll}\text{maximize} & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}\end{array}$$

with $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. Each row of $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ is a half-space: everything on one side of a flat boundary. The intersection of finitely many half-spaces is a polytope — a many-faced, flat-sided solid, a polygon in two dimensions — containing exactly the decisions that satisfy every constraint at once. This is the feasible region. The fact that makes the whole enterprise tractable is that if such a problem has an optimal solution, at least one optimal solution sits at a vertex of that polytope. Not floating in the interior, not stranded in the middle of a face.

CHECK THIS AGAINST YOUR INTUITION

If the objective's gradient points in some fixed direction and the feasible region is a flat-sided solid, is there any way to keep improving the objective without eventually running into an edge or a corner?

There is not, and the reason is that a linear objective has no curvature anywhere to create an interior peak. Suppose an optimal $\mathbf{x}^*$ were not a vertex. Then it lies strictly inside some line segment contained in the feasible region, so $\mathbf{x}^* = \frac{1}{2}(\mathbf{u} + \mathbf{v})$ for two feasible points $\mathbf{u} \neq \mathbf{v}$, and $\mathbf{c}^\top \mathbf{x}^* = \frac{1}{2}(\mathbf{c}^\top \mathbf{u} + \mathbf{c}^\top \mathbf{v})$. At least one endpoint must be at least as good as the midpoint, so you can slide along the segment without losing anything and repeat the argument on the lower-dimensional face you land in. Keep going and you arrive at a corner. If the objective happens to run exactly parallel to a face, an entire edge or facet ties for the optimum — but a vertex still attains it, which is all the algorithm needs.

That reduces an infinite search to a finite one, and the reduction alone is not enough. A problem in n variables with m inequalities can have on the order of $\binom{n+m}{m}$ vertices, which for a few hundred variables is a number with no physical meaning. Enumerating corners is hopeless. The trick is to visit only the corners that are getting better.

A Refinery in Two Dimensions

Take a small refinery deciding monthly output of a gasoline blend, $\mathbf{x}_1$, and a diesel blend, $\mathbf{x}_2$, in thousands of barrels per day. The contribution margin is 30 dollars per unit of gasoline and 20 per unit of diesel. Crude supply limits throughput, with gasoline consuming twice the crude per unit of output; refining hours limit combined output; and gasoline storage caps $\mathbf{x}_1$ at 40:

$$\begin{array}{llll}
\text{maximize} & 30x_1 + 20x_2 & & \\
\text{subject to} & 2x_1 + x_2 \leq 100 & (\text{crude}) & \\
& x_1 + x_2 \leq 80 & (\text{refining hours}) & \\
& x_1 \leq 40 & (\text{storage}) & \\
& x_1, x_2 \geq 0 & &
\end{array}$$

The feasible region is a pentagon with five vertices, and because the region is bounded and non-empty, one of them is optimal. Since there are only five, they can be listed outright:

Vertex	Crude used	Hours used	Profit
(0, 0)	0	0	0
(40, 0)	80	40	1200
(40, 20)	100	60	1600
(20, 60)	100	80	1800
(0, 80)	80	80	1600

The winner is **(20, 60)**, worth 1,800 dollars. Crude and refining hours are both exactly binding there, while storage is slack: the plan uses only 20 of the 40 units of gasoline capacity available. The geometry says the same thing more compactly. The iso-profit line $30x_1 + 20x_2 = 1800$ has slope $-\frac{3}{2}$, the crude boundary has slope -2 and the hours boundary -1 . Because the objective's slope lies strictly between the two, sliding the profit line outward pins it last against the single point where those two boundaries cross. The storage cap also cuts off **(50, 0)**, which crude and the axis would otherwise have made a vertex.

Enumeration works here because the pentagon has five corners. Dantzig's method never enumerates. Starting from the origin it admits gasoline first, since **30** is the steeper reward, and pushes x_1 until something stops it — storage does, at $x_1 = 40$, giving **(40, 0)** and a profit of 1,200. Diesel then enters and rises until crude binds at **(40, 20)**, worth 1,600. At that corner the algorithm discovers that backing gasoline away from its storage cap pays: sliding down the crude boundary trades one unit of gasoline for two of diesel, a swing of $-30 + 40 = +10$ per unit, until refining hours run out at **(20, 60)**. Now every direction out of the corner is a loss, and the search halts. Three pivots, four corners touched, and **(0, 80)** never examined at all.

Pivots, Tableaux, and Shadow Prices

Stated properly, the mechanics are bookkeeping on a system of equations. Add a slack variable to each inequality so that $Ax + s = b$ with $x, s \geq 0$, giving m equations in $n + m$ non-negative unknowns. Choose m of those columns as the basis B ; the remaining n are fixed at zero, and the basic values follow

as $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$. Every such choice with $\mathbf{x}_B \geq \mathbf{0}$ corresponds to exactly one vertex of the polytope, which is the algebraic translation of the geometry: corners are bases. A tableau is just this system rewritten so the basic columns form an identity matrix, updated in place.

One iteration asks two questions. Which non-basic variable, if raised off zero, would improve the objective? That is the sign of the reduced cost

$$\bar{c}_j = c_j - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j,$$

and any $\bar{c}_j > 0$ in a maximization identifies a profitable edge to walk along. How far can it be raised before some basic variable hits zero? That is the ratio test,

$$t^* = \min_i \left\{ \frac{(\mathbf{B}^{-1}\mathbf{b})_i}{(\mathbf{B}^{-1}\mathbf{A}_j)_i} : (\mathbf{B}^{-1}\mathbf{A}_j)_i > 0 \right\},$$

and the row achieving the minimum names the variable that leaves the basis. Swap the two, update, repeat. If no $\bar{c}_j$ is positive the current vertex is optimal, and the reduced costs are the proof — a certificate, not a hunch, which is exactly what Stigler lacked. If some $\bar{c}_j > 0$ but no ratio is finite, the objective is unbounded along that edge. Ties in the ratio test mean degeneracy: several constraints meet at one corner, the pivot moves nowhere, and without an anti-cycling rule the algorithm can in principle loop forever between bases describing the same point.

The reduced-cost vector also carries the economics. Every linear program has a dual, and where the primal asks which output mix maximizes profit, the dual asks what each scarce resource is worth at the margin. Here it is a minimization over one price per constraint,

$$\begin{array}{ll} \text{minimize} & 100y_1 + 80y_2 + 40y_3 \\ \text{subject to} & 2y_1 + y_2 + y_3 \geq 30 \\ & y_1 + y_2 \geq 20, \quad y \geq 0, \end{array}$$

whose solution is $\mathbf{y} = (10, 10, 0)$ with objective $1000 + 800 + 0 = 1800$ — identical to the primal maximum, as strong duality guarantees [11]. Crude and refining hours bind and carry prices of 10 dollars each; storage is slack and prices at zero, by the rule called complementary slackness. The interpretation is operational: the refinery should pay up to 10 dollars for one more unit of crude, or 10 dollars for one more refining hour, and nothing whatever for more storage [11]. Both claims survive a direct check. Raise crude to 101 and the optimum moves to **(21, 59)**, worth 1,810; raise hours to 81 and it moves to **(19, 62)**, also 1,810.

WORK OUT THE SIGN

Why must a constraint with slack left over carry a shadow price of exactly zero, no matter what the rest of the problem looks like?

Because if you already have more of something than you are using, one additional unit changes nothing you are able to do, so its marginal value must be zero. That single observation is one of the quietly powerful facts in economics: in an optimized system, only genuinely scarce things command a price. It is not confined to refineries. A trader quoting a crack spread, a firm setting an internal transfer price for a shared facility, a utility charging more for peak-hour electricity — all are reasoning in shadow prices that a dual program computes mechanically. Modern solvers do this at a scale Dantzig's desk calculators could not approach, handling millions of variables in seconds for airline crew pairing, refinery blending, grid dispatch and network design [10], [9].

The Worst Case That Never Arrives

For all of that, the simplex method has a real theoretical flaw. In 1972 Victor Klee and George Minty built a family of deliberately distorted high-dimensional cubes — Klee–Minty cubes — on which simplex, following its usual improve-the-objective rule, is forced to visit every single vertex before reaching the optimum [7]. The vertex count of a cube grows exponentially in dimension, so the method has no polynomial-time worst-case guarantee. On adversarial input it can run essentially forever.

Nobody has ever met such an input by accident, and the reason is instructive. A Klee–Minty cube is a knife-edge construction: the whole exponential path depends on the constraint slopes being tuned so that each corner looks marginally better than the last in exactly the wrong order. Perturb the coefficients slightly and the pathology collapses. That intuition is what average-case analysis, and later the framework known as smoothed analysis, made precise — asking not how an algorithm behaves on the worst instance an adversary can design, but how it behaves on instances subjected to small random perturbation. Under that lens simplex is fast, which matches what practitioners see: pivot counts that scale with the number of constraints rather than with the number of corners, on problems whose corner counts are astronomical.

Before that reconciliation, the gap between “provably fast” and “fast in practice” produced a genuine Cold War drama. In 1979 a young Soviet mathematician, Leonid Khachiyan, published a four-page paper showing that a different technique — the ellipsoid method, which traps the feasible region in a shrinking sequence of ellipsoids instead of hopping between vertices — solves any linear program in provably polynomial time [8]. It made the front page of the *New York Times* under the headline “A Soviet Discovery Rocks World of Mathematics” [8], partly because computer scientists had wondered for a decade whether such a bound existed at all, partly because it landed inside a contest over scientific prestige. The irony was that the ellipsoid method proved slow in practice. It settled a question of principle without producing a better tool.

The better tool arrived in 1984, when Narendra Karmarkar at AT&T Bell Labs published a genuinely fast polynomial-time method — an interior-point method, cutting diagonally through the middle of the polytope rather than crawling along its edges [6]. AT&T patented the algorithm in 1988 and sold it as a

proprietary system called KORBX, igniting a still-remembered argument over whether an algorithm should be patentable at all [6]. The patent lapsed in 2006. Industrial solvers now ship both families side by side, because each wins on different problem structures.

Property	Simplex	Interior-point
Path taken	Along edges	Through the interior
Worst case	Exponential (1972)	Polynomial (1984)
Exact vertex output	Yes, directly	Needs a crossover step
Origin	U.S. Air Force, 1947	AT&T Bell Labs, 1984

Where This Leaves Us

The socialist calculation debate of the 1920s and 1930s, in which Ludwig von Mises and Friedrich Hayek argued that central planners could never compute efficient prices without a market to generate them, was still an open wound in Soviet economics when Kantorovich turned up with a technique that computed exactly the sort of shadow prices Hayek insisted only markets could produce. Whether his method answered the challenge or merely relocated the same informational burden into another form is still argued over — a reminder that an elegant algorithm and a settled political question are not the same thing, even when they stand suspiciously close together.

What has not changed since 1947 is the underlying wager: that enough of the world’s planning problems can be captured well enough by straight lines and flat-sided regions for a machine walking from corner to corner to beat intuition and committee compromise. That wager keeps paying, not because reality is linear, but because a decent linear approximation solved exactly and quickly usually beats a perfect model solved never. Which leaves the harder puzzle. An algorithm with a proven catastrophic worst case is today among the most trusted pieces of infrastructure in the world economy, carrying real money daily on the strength of eighty years of instances that happened not to be adversarial. The guarantee we have is not the guarantee we rely on. If practical trust can be earned this thoroughly without a worst-case bound, it is worth asking what the bound was ever really doing for us — a question that returns almost verbatim in arguments about machine learning systems, which arrive with no worst-case guarantees at all.

REFERENCES

- [1] G. B. Dantzig, “The Diet Problem,” *Interfaces*, vol. 20, no. 4, 1990; see also “Stigler diet,” Wikipedia, accessed Aug. 2026. Available: https://en.wikipedia.org/wiki/Stigler_diet
- [2] “George Dantzig (1914–2005),” MacTutor History of Mathematics, University of St Andrews. Available: https://mathshistory.st-andrews.ac.uk/Biographies/Dantzig_George/
- [3] “Air Force Salutes Project SCOOP,” *ORMS Today*, INFORMS, Dec. 2007. Available: <https://www.informs.org/ORMS-Today/Archived-Issues/2007/orms-12-07/Air-Force-Salutes-Project-SCOOP>

- [4] S. I. Garille and S. I. Gass, “Stigler’s Diet Problem Revisited,” *Operations Research*, vol. 49, no. 1, pp. 1–13, 2001. Available: <https://pubsonline.informs.org/doi/pdf/10.1287/opre.49.1.1.11187>
- [5] I. Boldyrev and T. Düppe, “Programming the USSR: Leonid V. Kantorovich in context,” *The British Journal for the History of Science*, Cambridge University Press; see also “Kantorovich, Leonid V.,” INFORMS History of O.R. Excellence. Available: <https://www.cambridge.org/core/journals/british-journal-for-the-history-of-science/article/programming-the-ussr-leonid-v-kantorovich-in-context/4BFoFoD89079DD94AF595EA25A991299>
- [6] “Karmarkar’s algorithm,” Wikipedia, accessed Aug. 2026. Available: https://en.wikipedia.org/wiki/Karmarkar%27s_algorithm
- [7] V. Klee and G. J. Minty, “How good is the simplex algorithm?,” in *Inequalities III*, Academic Press, 1972; see also “Klee–Minty cube,” Wikipedia, accessed Aug. 2026. Available: https://en.wikipedia.org/wiki/Klee%E2%80%93Minty_cube
- [8] “Leonid Khachiyan,” Wikipedia, accessed Aug. 2026. Available: https://en.wikipedia.org/wiki/Leonid_Khachiyan
- [9] C. Barnhart et al., “Airline Crew Scheduling: Models and Algorithms,” and related large-scale crew-pairing literature. Available: https://ira.lib.polyu.edu.hk/bitstream/10397/94584/1/Wen_Airline_Crew_Scheduling.pdf
- [10] Gurobi Optimization, “Top Real-World Applications of Linear Programming.” Available: <https://www.gurobi.com/resources/faq/top-real-world-applications-of-linear-programming/>
- [11] B. A. McCarl and T. H. Spreen, “Duality in Linear Programming,” lecture notes, Texas A&M University, 2020. Available: <https://agecoresearch.tamu.edu/mccarl/wp-content/uploads/sites/4/2023/12/new04.pdf>; MIT 15.053 course notes, “Duality in Linear Programming,” Ch. 4. Available: <https://web.mit.edu/15.053/www/AMP-Chapter-04.pdf>

Daily Learning No. 002 · Operations Research · Friday, August 14th, 2026 · Sources are listed above; every load-bearing figure traces to a numbered reference.